

PRC Data Challenge 2026 — Taxi-Out Time Prediction

This repository contains the complete solution developed by the **joyous-rainbow** team for the PRC Data Challenge 2026, organised by the EUROCONTROL Performance Review Commission and the OpenSky Network, focused on predicting departure taxi-out times at major European airports.

Team & Acknowledgments

This repository is the official entry of the **joyous-rainbow** team for the data challenge.

Team members:

- **Rade Kačar** — University of Belgrade, Faculty of Transport and Traffic Engineering
- **Darko Čulibrk** — MTEL Banja Luka

We thank the challenge organizers and the open-source community for providing the tools and datasets that made this solution possible. We also acknowledge team **kind-mango**, whose publicly documented work inspired two ideas used here (see [Attribution of ideas](#)).

Overview

Task. Predict

$$\text{TAXITIME_SEC_mvt} = \text{MVT_TIME_UTC_mvt} - \text{BLOCK_TIME_UTC_mvt}$$

for **344,841 departures** at ten major European airports in January and July 2026, training on **4.17 million movements** from 2025.

At a high level, the project does the following:

1. Downloads hourly METAR observations for the ten airports from the Iowa Environmental Mesonet archive.
2. Computes stand-to-runway distances from OpenStreetMap taxiway geometry (optional).
3. Builds about fifty features per departure: Network Manager timestamp differences, categorical descriptors, congestion measures, airport regime, local time and weather.
4. Separates departures into regular flights and flights without a Network Manager record.
5. Trains a LightGBM model with linear leaves for regular flights and three special models for flights without an NM record.
6. Validates on January and July 2025, retrains on the full year and writes a checked submission file.

Central finding

The error is not spread evenly across flights. **2.9 % of departures produce 71 % of the squared error**, and they split into two groups with different causes. Most of the work went into identifying those causes rather than into building a larger model.

Group	What it is	Share of flights	Share of error	RMSE in group
C — regular	everything recorded properly	97.1 %	35.0 %	195 s
A — no Network Manager record	all NM columns empty	1.56 %	49.3 %	1,822 s
B — late off-block record	NM logs pushback ~25 min late	1.32 %	15.6 %	1,115 s

Shares are measured in the final model. Group B started at 1,680 s and fell by a third once the model was allowed to extrapolate; group A carries half the remaining error and is the hard limit of this task.

Group A. These departures were never matched to a EUROCONTROL Network Manager record — `AOBT_3_flt`, `EOBT_1_flt`, `LOBT_flt`, market segment and flight type are all missing at once. At some airports the airport system then wrote the scheduled off-block time into the block-time field. At Rome (LIRF) this happens for 51.4 % of such rows, so the computed taxi-out does not measure taxiing at all — it measures the delay of the flight, reaching values of up to 24 hours.

Group B. Here the records exist, but the Network Manager off-block time lags the airport one by a median of 25 minutes. The aircraft has left the stand while the network still considers it parked. These flights cluster strongly in time: the daily share varies 5.8× more than chance would produce, and the hour-to-hour correlation is 0.556 — multi-hour regimes, most likely de-icing in winter and flow restrictions in summer.

Pipeline Overview

A pipeline through this repository looks like this:

1. Compute stand-to-runway distances (optional)

- `osm_distances.py` queries OpenStreetMap via the Overpass API for parking positions, taxiways and runways, and writes `cache/stand_dist.parquet`.
- `check_osm.py` checks how well OSM stand names match the stand identifiers in the challenge data.

2. Main data processing and modeling

- `taxiout_final.py` is the central script. It runs on its own and orchestrates the entire workflow from raw data to the submission file. If `cache/stand_dist.parquet` exists it is used; otherwise the pipeline works without it.

3. Download weather

- `taxiout_final.py` fetches METAR observations for the ten airports and caches them in `cache/`.

4. Build features

- Features are computed in parallel across the twelve monthly files and cached. The cache carries a version marker and is rebuilt automatically when the feature code changes.

5. Train and validate

- The main LightGBM model (linear leaves) is trained on departures with a valid NM record; three special models are trained on group A.
- January and July 2025 are held out for validation; the remaining ten months are used for training.
- Optionally, a 100-trial Optuna hyperparameter search is run (`RUN_OPTUNA = True`).

6. Retrain and generate submission

- Models are retrained on the full year 2025 and applied to the 2026 ranking set.
- `<team-name>_v<n>.parquet` is written after five validity checks: row count, identifier match, column names, missing values and infinite values.

7. Figures

- `figures.py` produces the figures used in the write-up stored in `docs/`.

Repository Structure

Top-level files

File / Folder	Role
<code>README.md</code>	Project documentation.
<code>LICENSE</code>	GNU General Public License v3.0 only, as required for prize eligibility.
<code>taxiout_final.py</code>	Complete pipeline: METAR download, feature engineering, main and special models, validation, retraining and submission file generation.
<code>osm_distances.py</code>	Computes stand-to-runway distances from OpenStreetMap (Overpass API) and writes <code>cache/stand_dist.parquet</code> .
<code>check_osm.py</code>	Checks how well OSM stand names match the challenge data.
<code>figures.py</code>	Generates the figures used in the write-up.
<code>docs/</code>	Write-up and figures.
<code>cache/</code>	Cached weather, features and intermediate artifacts (created at run time).

`cache/`

The `cache/` directory is created by the pipeline and makes re-runs fast.

File / Folder	Role
<code>stand_dist.parquet</code>	Stand-to-runway distances produced by <code>osm_distances.py</code> (optional).
METAR cache	Hourly weather observations for the ten airports, downloaded by <code>taxiout_final.py</code> .
Feature cache	Per-month feature tables with a version marker; rebuilt automatically when the feature code changes.

Other generated artifacts

File	Role
<code>optuna_taxiout.db</code>	Optuna study state; allows the hyperparameter search to be interrupted and resumed.
<code><team-name>_v<n>.parquet</code>	Versioned submission file, written after the validity checks pass.

Method

Two models, applied to disjoint sets of rows.

Main model — LightGBM for departures with a valid NM record

About fifty features in six families:

Family	Content
NM timestamp differences	Differences between NM timestamps; $\text{MVT_TIME} - \text{AOBT_3}$ alone carries 32 % of the gain.
Categories	Airport, stand, runway, aircraft type, operator, market segment, wake category.
Congestion	Movement counts in $\pm 10/30/60$ min windows, per airport and per runway; runway share of traffic as a proxy for configuration; arrival taxi-in in the previous 30 and 60 minutes (survives into the ranking set, since only departure taxi-out was blanked).
Airport regime	Mean $\text{MVT_TIME} - \text{AOBT_3}$ over recent departures.
Time and weather	Local time with cyclical encoding; METAR observations.
Geometry	Stand-to-runway distances computed from OpenStreetMap taxiway geometry.

Linear leaves. One setting mattered more than every feature combined. A tree returns a constant in each leaf, so the ensemble cannot predict a value larger than the largest one it saw in training — and that is exactly where half the error lives. LightGBM's `linear_tree` makes the leaves linear functions and removes that ceiling: local RMSE 364.0 $\rightarrow$ 338.5, error on long flights 1,666 $\rightarrow$ 1,181, and on the official score a single 31.4 s gain, larger than everything else put together.

Two measures usually recommended alongside such a model turned out to be harmful here: capping predictions at the per-airport maximum (+19.7 s) and dropping the hardest airport from training (+14.6 s). Both cripple a model that can now recognise genuinely long flights.

Special models — three models for group A

With every NM column missing, one usable variable remains: the delay of the take-off against the schedule, whose correlation with the target in this group is 0.854.

Model	Description	RMSE in group A
Delay curve	Per-airport average of a cubic polynomial and a hinge at three hours (flat for small delays, then rising roughly one-to-one once the delay exceeds a few hours).	1,908 s
Trees	LightGBM with linear leaves.	1,924 s
Curve + trees average	Simple average of the two.	1,863 s
Mixture	A classifier estimates the probability that a row belongs to the broken regime (AUC 0.956, probability shrunk by 0.8) and splits the prediction between the delay and a separate model for the remaining rows.	1,860 s
All three combined		1,822 s

They fail in different places, so the combination wins.

Validation

January and July 2025 are held out and the remaining ten months are used for training, so the local split mirrors the seasonal composition of the ranking set. All history-based features look strictly backwards, and all per-stand statistics are computed on the training part only.

Results

Configuration	Local RMSE	Official RMSE
Constant (training median)	520	—
LightGBM alone	531	—
+ special treatment of group A (straight line)	382	—
+ training on flights up to 3 hours	372	—
+ arrival taxi-in and airport regime	371	—
+ cubic curve for group A	364.2	337.5
+ Optuna hyperparameters	361.3	333.1
+ OpenStreetMap stand-to-runway distances	361.0	333.7
+ linear leaves (<code>linear_tree</code>)	331.7	301.8
+ curve-and-trees average for group A	330.5	299.9
+ classifier mixture for group A	324.1	299.2

LightGBM alone scoring worse than a constant is a consequence of the metric: the model learns typical values and misses the tail catastrophically, while a constant at least does not make those cases worse. Only once the tail is handled separately does the model's strength show.

On regular flights the model reaches **RMSE 194.7 s** and **MAE 137.7 s**, better than the overall score of the leading team, so the entire gap to the top of the leaderboard sits in a few thousand atypical rows.

Local validation versus official score

Seven paired measurements give a result that is useful beyond this competition. The official score is consistently better than the local one, by 24.9 to 30.6 seconds, and the gap widens slightly as the model improves. The explanation is the composition of the tail: the 2026 data contains fewer extreme records than January and July 2025. The better the regular flights are handled, the more of the remaining error comes from the tail, and the more visible the difference between the two years becomes.

The practical consequence: changes worth less than about two seconds locally did not transfer reliably — the OpenStreetMap distances improved the local score by 0.3 s and worsened the official one by 0.6 s. Changes above two seconds transferred every time, and transferred amplified. That is the empirical noise floor for this problem, and it is higher than

one would expect from 344,841 rows, precisely because a few thousand atypical rows carry half the score.

What did not work

Eleven approaches commonly recommended for heavy tails or for enriching the model were measured and rejected. Reporting them is as useful as reporting the successful steps.

Approach	Change in RMSE	Why it failed
Clipping predictions at 1 hour	+178 s	The extremes exist in the ranking set too.
Per-airport prediction cap	+19.7 s	Cripples a model that can now recognise long flights.
Dropping the hardest airport from training	+14.6 s	Loses signal that holds elsewhere too.
Robust (Huber) regression for group A	+16 s	The extremes there are correct data, not noise.
Blending towards a typical value	+6.4 s	Under squared loss the optimum is the conditional mean.
Runway service rate and expected queue wait	+1.8 s	The same signal is already carried by the NM time difference.
Huber loss in the main model	+1.8 s	RMSE punishes quadratically; softening works against it.
Two-stage model with a tail classifier	+0.7 s	The classifier is not confident enough (mean probability 0.43).
Binned conditional mean for group A	+0.6 s	The curve already captures the shape.
Congestion normalised per airport	+0.5 s	The model already has the airport as a category.
Weather as a whole (METAR)	no change	See below.

Congestion. Four independent attempts to measure congestion — per-runway movement counts, arrival taxi-in, an airport regime indicator and a service-rate queue estimate — all landed between -0.8 and $+1.8$ seconds. The reason is consistent: the difference between take-off and the recorded off-block time already contains the result of waiting. Congestion

explains *why* an aircraft waits; the model does not need the explanation, it already has the measurement.

Weather. Measured as a whole, METAR features changed nothing. But when the visibility column was accidentally lost during a folder migration and the pipeline re-ran without it, the score moved by 4.5 seconds. Visibility is the one weather variable that carries information here, and averaging it together with temperature and precipitation had hidden that. It makes physical sense: reduced visibility triggers low-visibility procedures that slow every ground movement in that period, while temperature and snow act at the level of a day and affect only some flights.

Reproducing

```
bash

pip install pandas numpy pyarrow lightgbm optuna requests

# 1. stand-to-runway distances (optional, writes cache/stand_dist.parquet)
python osm_distances.py

# 2. everything else: METAR download, features, validation, submission file
#     set DATA_DIR and TEAM_NAME at the top of the file first
python taxiout_final.py
```

Hyperparameters found by a 100-trial Optuna search are hard-coded as defaults, so the reported result is reproducible without repeating the search. Set `RUN_OPTUNA = True` to run it again; it takes several hours and stores its state in `optuna_taxiout.db`, so it can be interrupted and resumed.

Re-runs are fast because features and weather are read from `cache/`.

Data and external sources

Challenge data (movement records joined with EUROCONTROL Network Manager records) is supplied by the organisers and is not redistributed here; obtain it through your own team bucket.

External sources, all openly licensed:

Source	Content	Licence
Iowa Environmental Mesonet	ASOS/METAR archive, hourly observations for the ten airports	Public domain
OpenStreetMap contributors	Parking positions, taxiways and runways, via the Overpass API	ODbL

Attribution of ideas

The solution is original and written from scratch. Two ideas came from the publicly documented work of other teams in this challenge and were implemented independently here: LightGBM's linear leaves, and the classifier-weighted mixture for the fallback regime (team **kind-mango**, whose repository documents both along with a long list of measured failures). Their implementation differs from ours in an important respect — that team deliberately avoids the Network Manager off-block field altogether, while this solution uses it.

On that field. `(MVT_TIME.UTC_mvt - AOBT_3_flt)` is this model's strongest single feature, and it approximates the blanked block time. The challenge brief cautions against it; asked directly, the organisers stated on 2026-09-18 that no such restriction exists and that even reconstructing off-block times from open trajectory data is acceptable, the model being intended for post-operations analysis rather than tactical use. The field is therefore used here, and this note records the choice openly.

Licence

GNU General Public License v3.0 only, as required for prize eligibility. See `LICENSE`.

OpenStreetMap-derived data is subject to the Open Database License; METAR observations are in the public domain.